

Part 5: Visualization

Summary

- This Part covers tools for visualizing data in R.
- Powerful visualization techniques are critical to discovering and understanding the relationships between variables.

Example Data Set

- First, we will revisit the data set we considered in the previous Part, and the steps needed to properly format the data:

```
> fulldata = read.table("q1_2017_all.csv", sep=",", header=T, quote="")
> fulldata$Date = as.Date(as.character(fulldata$Date), format="%Y%m%d")
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)], fromLast=TRUE),]
> fulldata$McapRank = factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank = factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank = factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank = factor(fulldata$PriceRank, ordered=TRUE)
```

- The entire data set is a bit large to work with conveniently, so we are going to restrict consideration to 1000 equities:

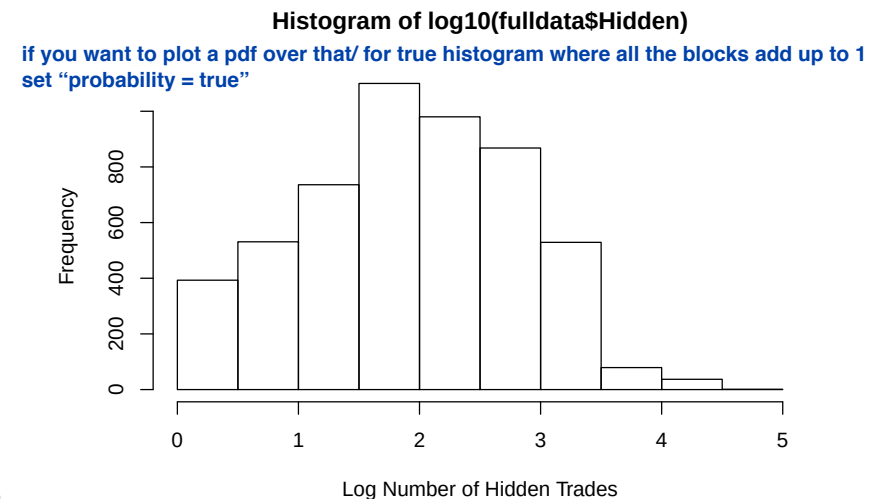
```
> tickkeep = unique(fulldata$Ticker)[seq(1,
+   length(unique(fulldata$Ticker)), length=100)]
> fulldata = fulldata[fulldata$Ticker %in% tickkeep,]
```

Classic Plotting Functions in R

- Prior to moving on the more advanced visualization functions, it is important to understand the basic plotting capabilities of R.
- We saw the use of `boxplot()` in a previous Part. A similar view of the shape of a distribution can be obtained using a **histogram**, formed using the function `hist()`:

```
> hist(log10(fulldata$Hidden),
+   xlab="Log Number of Hidden Trades")
```

- The title on the plot can be changed using the `main` argument.
- The number of bins in the histogram can be altered using the `breaks` argument.



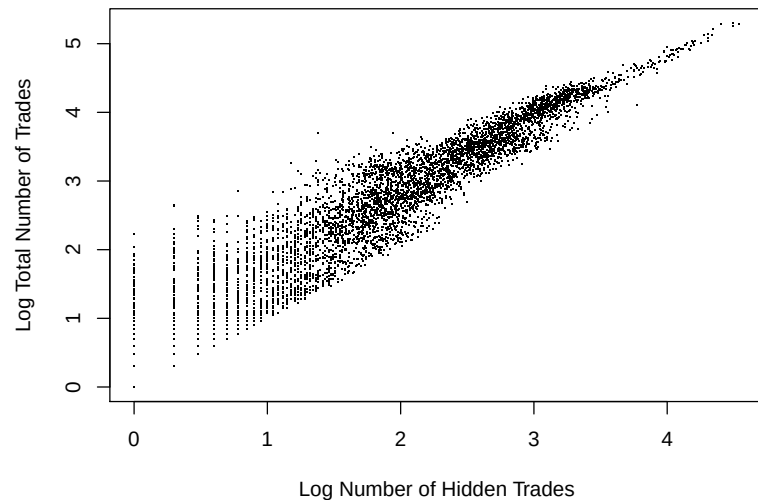
- 1 **Scatter plots** are ubiquitous, as well. They are created simply in R using
- 2 the `plot()` function:

```
> plot(log10(fulldata$Hidden),
+      log10(fulldata$TradesForHidden),
+      xlab="Log Number of Hidden Trades",
+      ylab="Log Total Number of Trades", pch=".")
```

default dots are hard to read, so set it to "."

- 3 The argument `pch` changes the plotting character. In a case such as this,
- 4 with so many dots on the plot, the default plotting character is too large.
- 5 Many other properties of the scatter plot can be adjusted using the `par()`
- 6 function. See `help(par)` for more information.

If labels do not show up, use `dev.off()` to clear current graphs in the workspace



1 Introduction to ggplot2

part of the larger package "tidyverse"

- 2 The package `ggplot2` contains a great deal of valuable data visualization tools. This package can be installed, along with other useful packages
- 3 for working with large and complex data sets, via the `tidyverse` meta-
- 4 package:

```
> install.packages("tidyverse")
```

Very useful package

- 6 The collection of packages in `tidyverse` comprise "a collection of R packages that share common philosophies and are designed to work together."
- 7
- 8 – tidyverse.org

- 1 The `ggplot2` package implements a **grammar of graphics**, where plots are
- 2 created by combining **geoms**.
- 3 The syntax is a little awkward at first, but this approach yields a great deal
- 4 of flexibility, and a great deal of power in exploring features in a data set.
- 5 A plot is initialized by using the `ggplot()` function:

```
> ggplot(data = fulldata)
```

you have to load your data into dataframe to use `ggplot`

- 1 To the basic plot, one adds the “aesthetic” via the `mapping` option. An
- 2 aesthetic is a generic concept for the visual features of a plot. In this case, we
- 3 are specifying that the variable on the horizontal axis is `Hidden`, and the
- 4 variable on the vertical axis is `Cancels`:

```
> ggplot(data = fulldata, mapping =
+       aes(x=Hidden, y=Cancels))
```

- 5 We can now add onto this base to create plots with complex features. You
- 6 can, for instance, define

```
> baseplot = ggplot(data = fulldata, mapping =
+       aes(x=Hidden, y=Cancels))
```

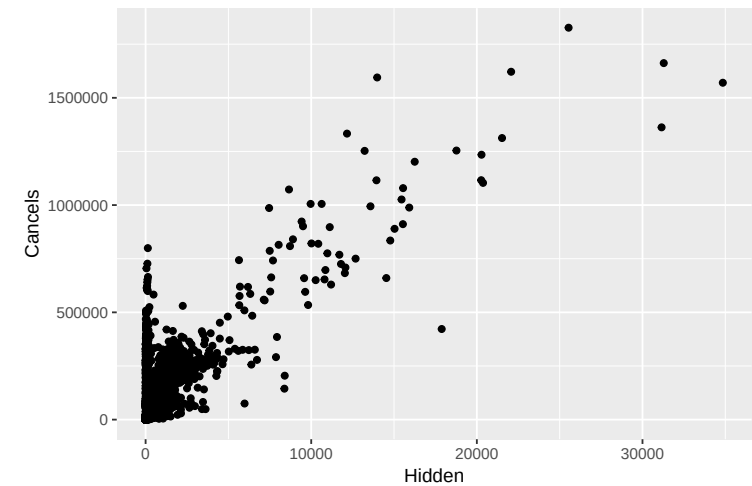
- 7 and add onto `baseplot` in what follows.

1 A Simple Scatter Plot

- 2 Consider the following syntax:

```
> baseplot + geom_point()
```

- 3 Note how the `geom_point()` function is used to add a scatter plot onto
- 4 the current figure, as stored in `baseplot`.

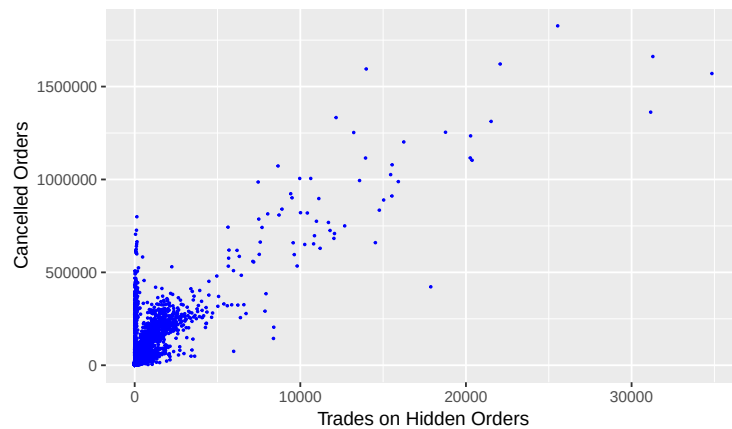


- 1 Now we will work to improve the appearance of this plot.

```
> baseplot +
+   geom_point(size=0.3, color="blue") +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017")
```

Cancelled Orders versus Hidden Orders

Daily, First Quarter of 2017



1

1 **Exercise:** Describe the changes that were made to improve the plot.

2 **1. better axis labels**

3 **2. Informative titles**

4 **3. Added color**

5 **4. Made plotting symbols smaller to increase "Seperation"**

6 _____

7 _____

8 _____

9 _____

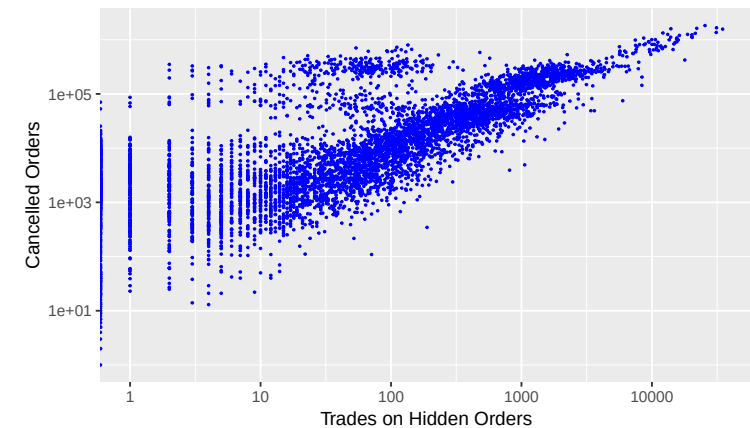
10

1 Next, we will convert the axes to the log scale:

```
> baseplot +
+   geom_point(size=0.3, color="blue") +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10()
```

Cancelled Orders versus Hidden Orders

Daily, First Quarter of 2017

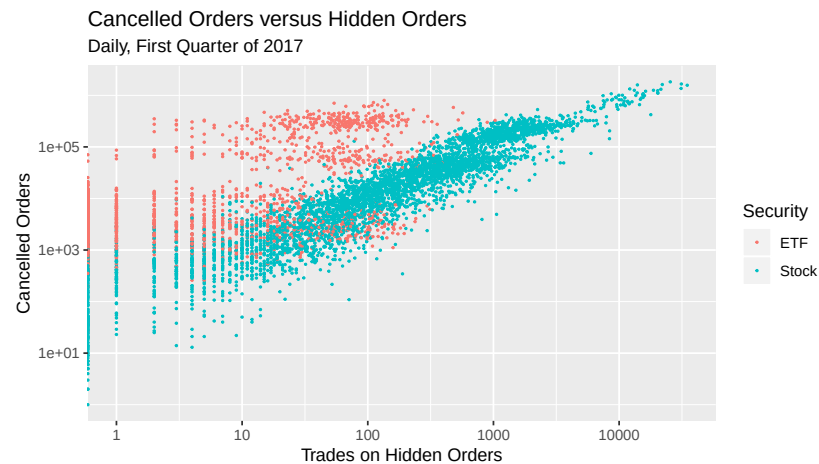


1

- 1 The `color` argument can be switched to vary with the levels of one of the
- 2 factors.

```
> baseplot +
+   geom_point(size=0.3, mapping=aes(color=Security)) +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10()
```

I want color to responds to Security

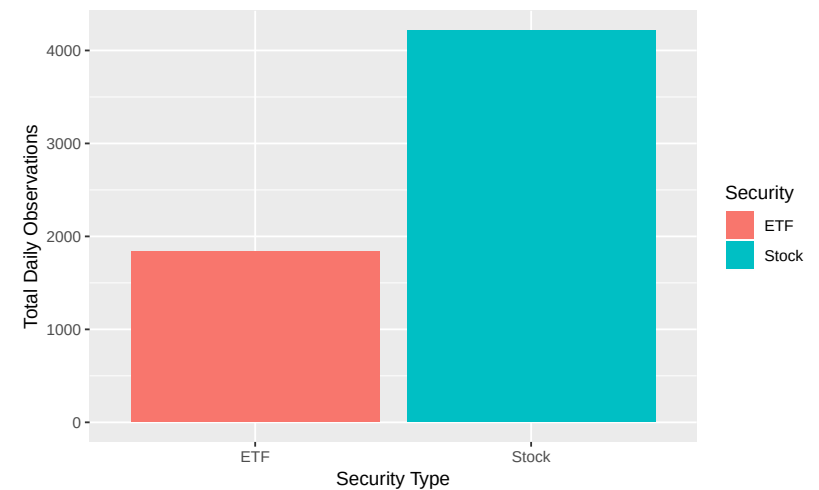


- 1 **Bar Plots**
- 2 Basic bar charts can be created to inspect the distribution of a factor, using
- 3 the `geom_bar()` function:

```
> ggplot(data=fulldata,
+        mapping=aes(x=Security, fill=Security)) +
+   geom_bar() +
+   labs(x="Security Type",
+        y="Total Daily Observations")
```

Specify fill colors by Security type

X: Security type, Y: amount



1 **Exercise:** Now that we have seen a few examples, try to explain the notion
2 of the “aesthetic.”

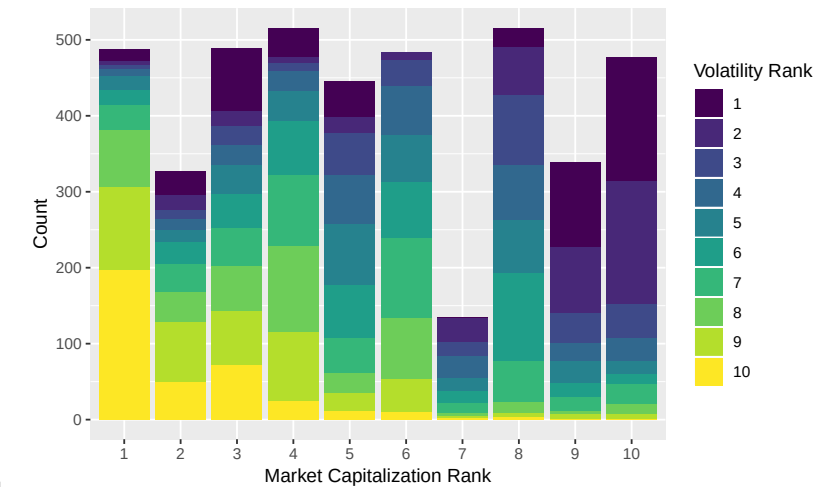
3	
4	
5	
6	
7	
8	
9	
10	
11	

1 For the next plot, we will restrict to data that do not have missing values
2 on McapRank, and are of type "Stock":

```
> fulldataStock = filter(fulldata,
+   Security == "Stock" & !is.na(McapRank))
```

3 We will adjust the aesthetic so that the “fill” color of the bars corresponds
4 to the level of VolatilityRank:

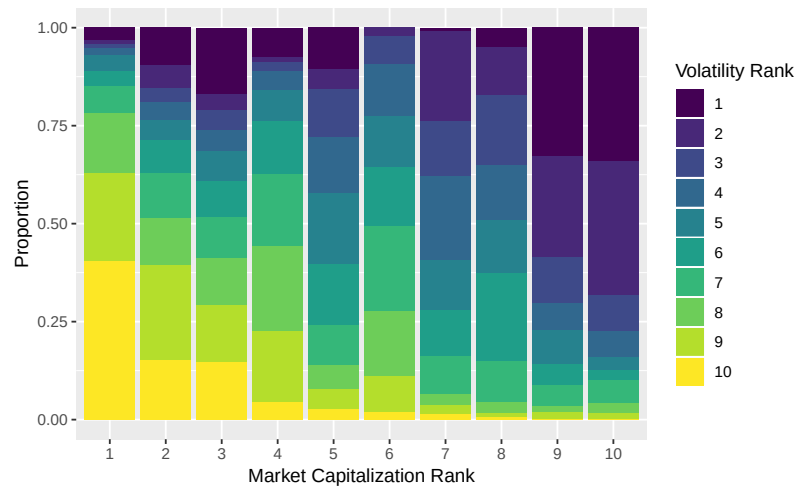
```
> ggplot(data=fulldataStock,
+       mapping=aes(x=McapRank, fill=VolatilityRank)) +
+   geom_bar() +
+   labs(x="Market Capitalization Rank", y="Count",
+        fill="Volatility Rank")
```



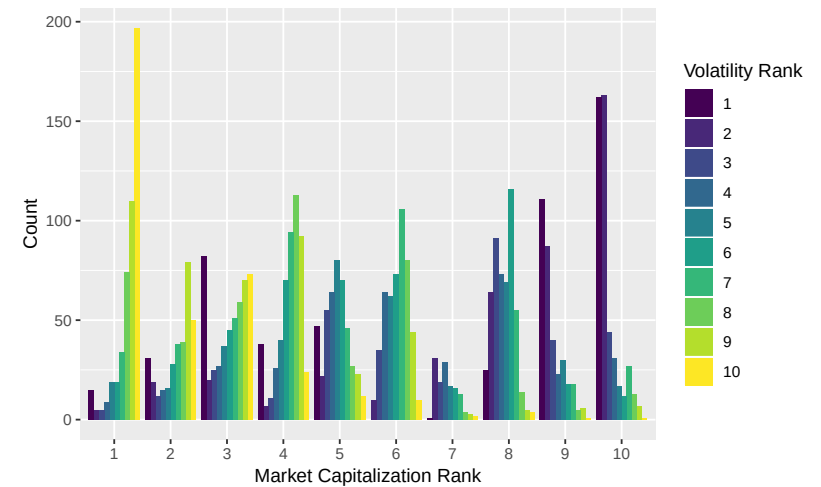
1 The bars in the previous plot are not of equal height because of the sub-
2 sampling we did to reduce to 1000 equities.

3 We can change vertical axis to proportion by setting `position="fill"`:

```
> ggplot (data=fulldataStock,
+         mapping=aes (x=McapRank, fill=VolatilityRank)) +
+         geom_bar (position="fill") +
+         labs (x="Market Capitalization Rank",
+              y="Proportion", fill="Volatility Rank")
```



1



1

1 By setting `position="dodge"`, the bars are arranged more like a his-

2 togram:

```
> ggplot(data=fulldataStock,
+   mapping=aes(x=McapRank, fill=VolatilityRank)) +
+   geom_bar(position="dodge") +
+   labs(x="Market Capitalization Rank",
+    y="Count", fill="Volatility Rank")
```

1 **Exercise:** Why would you not want to create a scatter plot to explore the re-

2 lationship between Market Capitalization Rank and Volatility Rank? Con-

3 sider how the function `geom_jitter()` can help in this regard.

4 A 10*10 grid of points, since Market Capitalization Rank and Volatility rank goes

5 from 1 to 10.

6 most of the data would be overlapping

7

8

9

10

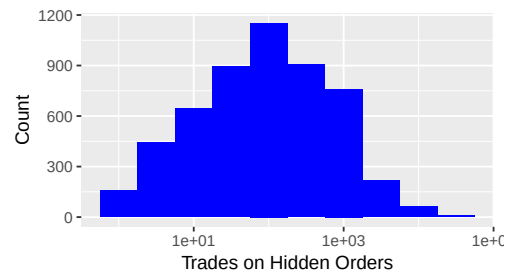
11

12

1 Histograms

2 The function `geom_histogram()` will create a histogram:

```
> ggplot(data=fulldata, mapping=aes(x=Hidden)) +
+   geom_histogram(binwidth=0.5, fill="blue") +
+   scale_x_log10() +
+   labs(x="Trades on Hidden Orders", y="Count")
```

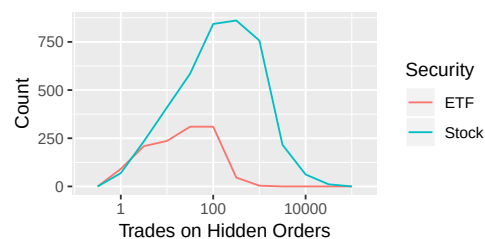


3

1 Overlapping histograms can be difficult to interpret (try!). An alternative

2 is to create **frequency polygons** using `geom_freqpoly()`:

```
> ggplot(data=fulldata,
+   mapping=aes(x=Hidden, color=Security)) +
+   geom_freqpoly(binwidth=0.5, fill="blue") +
+   scale_x_log10() +
+   labs(x="Trades on Hidden Orders", y="Count")
```



3

1 **Exercise:** Both `geom_histogram()` and `geom_freqpoly()` have an argument `binwidth`. Discuss the technical and practical effects of varying `binwidth`.

4 **In both cases, decreasing binwidth increases the number of “groups” the data are separated into, creating a “rougher” histogram/ frequency polygon**

6 **Setting binwidth too small leads to a less informative, less summarized view of the distribution**

7

8

9

10

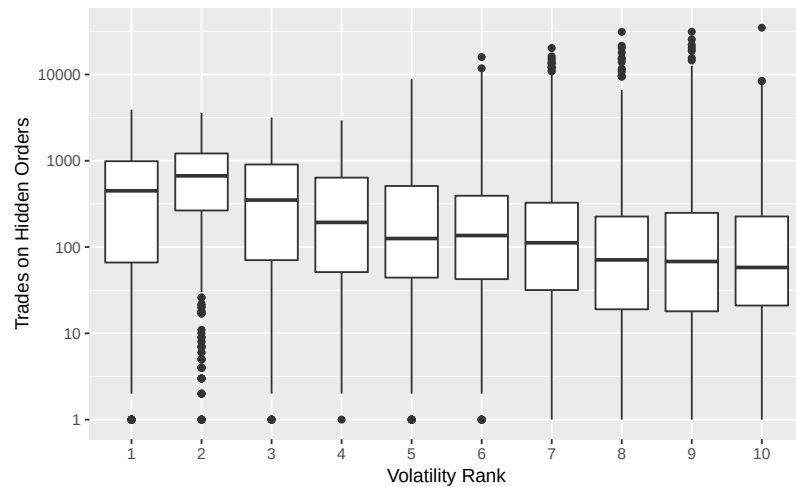
11

12

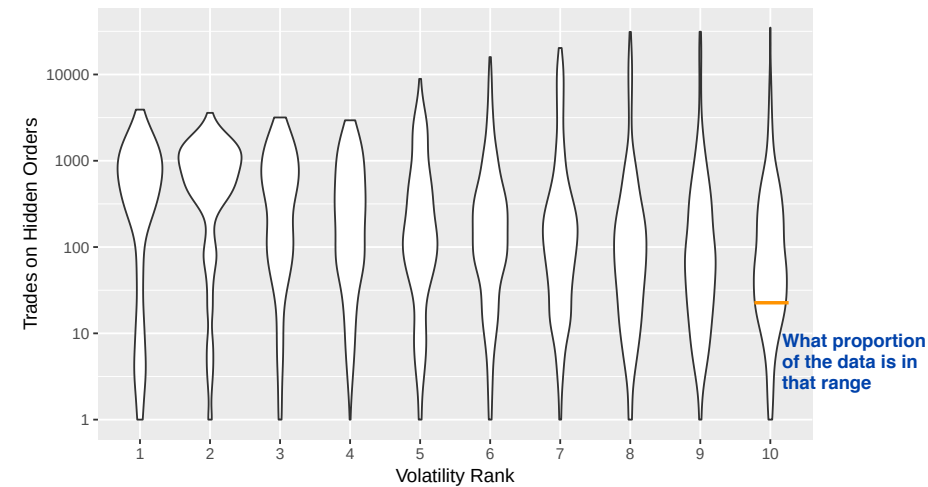
1 Boxplots

2 **Side-by-side boxplots** are also useful for comparing the distributions of variables over different values of a factor. For example,

```
> ggplot(data=fulldataStock,
+   mapping=aes(x=VolatilityRank, y=Hidden)) +
+   geom_boxplot() + scale_y_log10() +
+   labs(y="Trades on Hidden Orders",
+   x="Volatility Rank")
```



1



1

- 1 A **violin plot** shows the same type of information, with a little more detail.
- 2 The width of a “violins” is greater in areas where a greater proportion of
- 3 the observations lie.

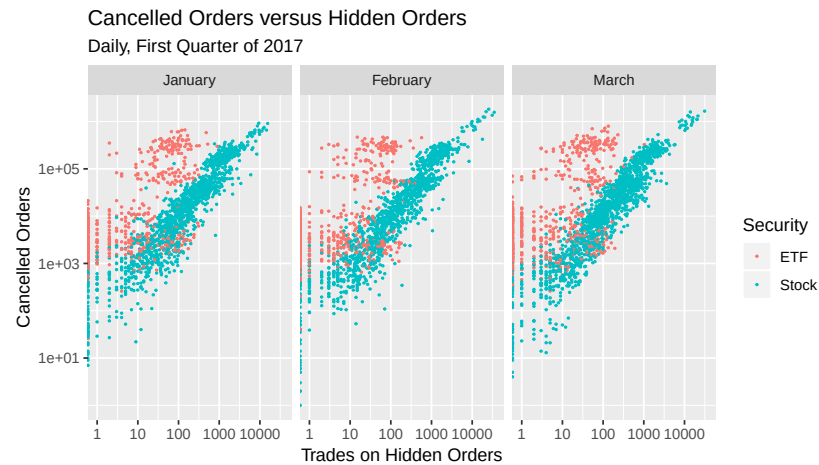
```
> ggplot(data=fulldataStock,
+       mapping=aes(x=VolatilityRank, y=Hidden)) +
+   geom_violin() + scale_y_log10() +
+   labs(y="Trades on Hidden Orders",
+        x="Volatility Rank")
```

1 Facets

- 2 **Facets** refer to an arrangement of plots on which one or more factors
- 3 **vary**. For example, return to our scatter plot example above, and use the
- 4 `facet_grid()` function to break up the plots by month:

```
> baseplot +
+   geom_point(size=0.3, mapping=aes(color=Security)) +
+   labs(x="Trades on Hidden Orders",
+        y="Cancelled Orders",
+        title="Cancelled Orders versus Hidden Orders",
+        subtitle="Daily, First Quarter of 2017") +
+   scale_x_log10() + scale_y_log10() +
+   facet_grid(.~factor(months(Date),
+                        levels=c("January", "February", "March")))
```

If there is no ordering of levels, `r` has no idea of the ordering of month
It will treat it as characters, ordering alphabetically



1

Two factors can vary:

```
> baseplot +
+ geom_point(size=0.3, color="red") +
+ labs(x="Trades on Hidden Orders",
+       y="Cancelled Orders",
+       title="Cancelled Orders versus Hidden Orders",
+       subtitle="Daily, First Quarter of 2017") +
+ scale_x_log10() + scale_y_log10() +
+ facet_grid(Security~factor(months(Date),
+                             levels=c("January", "February", "March")))
```

Security be across the rows, months be across the columns

See the labels across the top and along the side

1 **Exercise:** What happens in the previous example if the command

```
> facet_grid(.~months(Date))
```

2 is used instead? What about if use the following?

```
> facet_grid(months(Date)~.)
```

Without levels=c(), R will put them in alphabetical order. Not good

3

4

5

6

7

8

9



1

1 Time Series Plots

2 We will, of course, be working with a great deal of **time series data**. The

3 `geom_line()` function is useful:

I am plotting only one specific stock, so I'm using `filter`

```
> ggplot(data=filter(fulldata, Ticker=="AMD"),
+         mapping=aes(x=Date, y=Hidden)) +
+   geom_line(color="blue") +
+   labs(x="Date", y="Trades on Hidden Orders",
+        title="Change in Hidden Orders over Time",
+        subtitle="AMD, daily, first quarter of 2017")
```

Change in Hidden Orders over Time

AMD, daily, first quarter of 2017

